Policy gradient methods

Lecture 1: from gradient ascent to REINFORCE

Carlos Cotrini

ETH Zurich

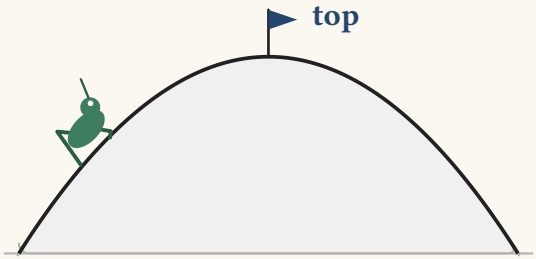
20 October 2026

1

Part 1

Gradient ascent

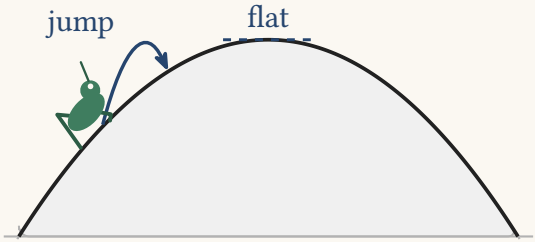
A grasshopper wants to reach the top of the hill



1 The grasshopper stands on the slope.

2 The top is the point of highest altitude: the place it wants to reach.

Keep jumping uphill until the ground is flat



Idea

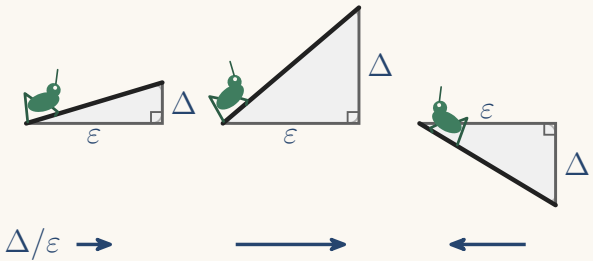
```
while not flat:
    jump in the direction
    that increases height
```

1 At the top the ground is flat: the grasshopper stops there.

2 Everywhere else it jumps uphill.

3 To know which way is uphill, it needs the gradient.

The gradient is the change in altitude per step

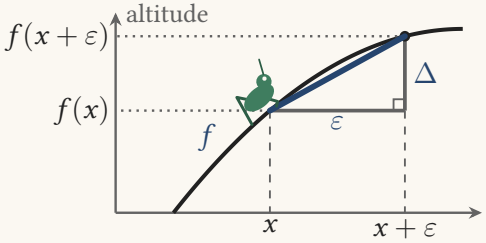


$$\text{Gradient} \approx \frac{\Delta}{\varepsilon} = \frac{\text{change in altitude}}{\text{step size}}$$

Here the step size is the probe step ε. The jump size α comes later.

1. ε (epsilon) is the small step we probe with. Δ (delta) is the altitude change over it.
2. The gradient is Δ/ε. We draw it as an arrow under each slope.
3. Steeper means a longer arrow. When the altitude drops, the arrow points left.

Python measures the gradient with a small step



function *location*

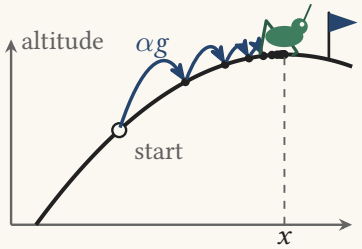
```
def grad(f, x):
    eps = 0.01          # small
    delta = f(x + eps) - f(x)
    grad = delta / eps
    return grad
```

1 `eps` is epsilon, the small step we probe with. The picture draws it large.

2 `delta` is the change in altitude over that step: `f(x + eps)` minus `f(x)`.

3 `grad` is `delta / eps`, the slope of the segment between the two points.

Gradient ascent measures, jumps and repeats




```
def gradient_ascent(f, x, alpha=0.1):
    g = grad(f, x)
    while abs(g) > 0.01:
        x = x + alpha * g
        g = grad(f, x)
    return x
```

1 `g` is the gradient at `x`. Flat means `g` is close to zero, here `abs(g)` at most 0.01.

2 `alpha` is how far we jump per unit of gradient: the step size of the climb.

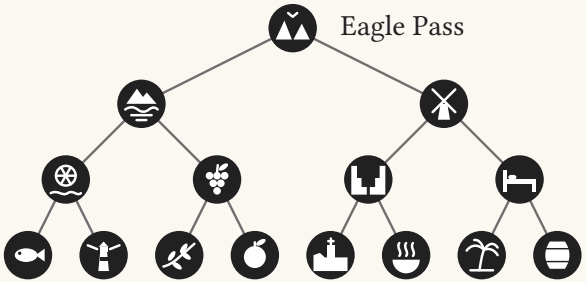
3 `eps` probes the ground, `alpha` sets the jump: they are different numbers.

2

Part 2

Mara’s road

A town of fifteen places joined by roads



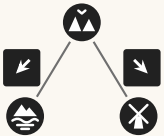
1 Fifteen places, joined by roads. A day follows one path down to the coast.

2 Play it yourself: scan the code or type the address.

`https://eth-cscs-hs26.github.io/CSCS-PAGE/viz/part2/courier-road/`

Mara plays her fiddle on the way down to the sea





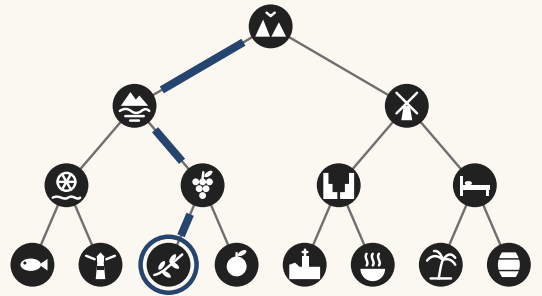


1 Mara is a street fiddler. In one day she walks from Eagle Pass down to the sea, through three places.

2 At every place the road forks: ↙ southwest or ↘ southeast. She picks one and walks on.

3 In every place she plays and the crowd drops coins in her hat. A place pays its usual coins, give or take one coin, whoever passes by. She wants as many coins as possible, but cannot know what a place will give until she has played there.

A simple strategy: play each path several times



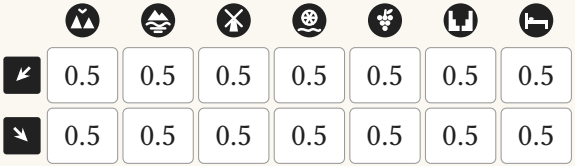
average coins of 5 days on each path

1 Luck makes the days differ. Play 5 days on each of the 8 paths and note the average of each.

2 Keep the best average. It finds the best path in 98% of the tries; one day per path only in 69%.

3 Trying every path works here, since there are only 8. One more place on a path doubles that to 16, and each further place doubles it again.

Mara follows a table of chances

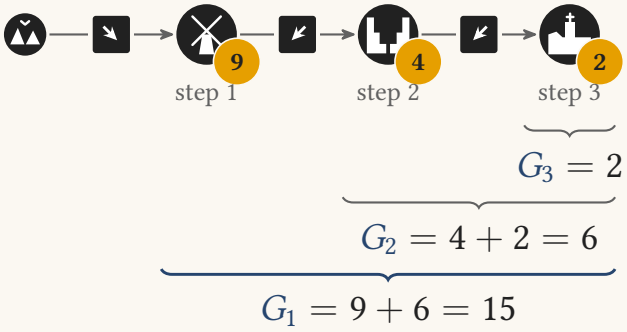


1 A table of chances: a column for each place where the road forks, a row for each road. Each column adds up to 1, and every cell starts at 0.5.

2 At each place s Mara takes road a with the chance in its cell, written $\pi(a \mid s)$. The table is her policy: her rule for choosing roads.

3 We use a method inspired by gradient ascent because it works well in practice on a wide variety of problems, and it generalises better than the simple strategy.

G_t is the sum of the coins from step t on



1 Gradient ascent needs an altitude. Ours is made of a day's coins, luck included.

2 G_t is the sum of the coins from step t to the end of the day.

3 G_1 is the whole day: here 15 coins.

The same table gives different days

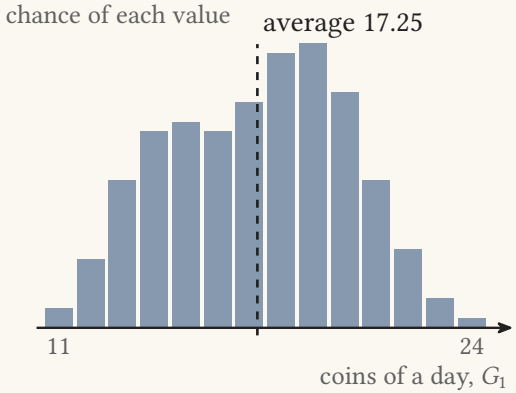
	step 1	step 2	step 3	G_1
day 1	9	4	2	15
day 2	7	7	3	17
day 3	8	5	1	14
day 4	9	3	3	15
day 5	10	3	6	19

1 One row is one day of the same table: its places and their coins.

2 What changes: the roads, decided by the table's chances, and the luck of each place.

3 So G_1 is random: a different number every day.

A random number cannot be maximised

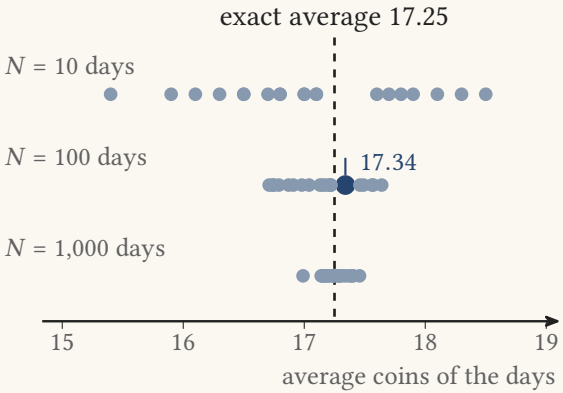


1 On many days the untouched table gives G_1 anywhere from 11 to 24 coins, some values more often than others.

2 The average of G_1 over all the days it can give is 17.25 coins.

3 A random number cannot be maximised: asking which table gives the biggest G has no answer.

More days bring the average closer to 17.25



1 Each dot is the average of N days in one of 20 runs. The line is the exact average of all possible days.

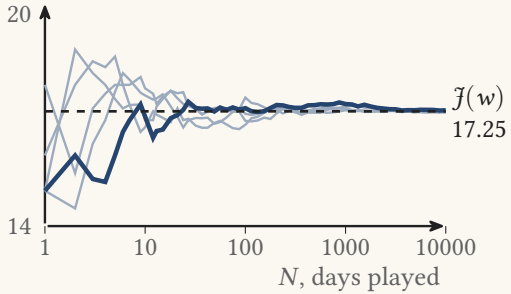
2 The dots miss the line by about 0.89 coins at $N = 10$, 0.28 at $N = 100$ and 0.09 at $N = 1{,}000$: the error shrinks as N grows.

3 The first run of 100 days averages 17.34: our estimate of the exact average from now on.

Our objective is the long-run average $\mathbb{E}[G_1]$

$$\mathbb{E}[G_1] = \lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} G_1^{(n)}$$

average coins of the first N days

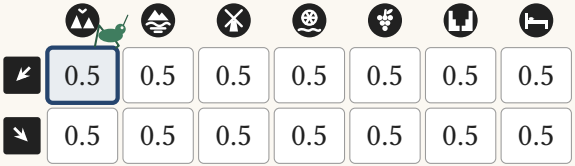


1 Let π_w be the policy of the table w. Play it on N days: the n-th day pays $G_1^{(n)}$.

2 The average of $G_1^{(1)}, \ldots, G_1^{(N)}$ settles as N grows, at a number we call $\mathbb{E}[G_1]$.

3 $\mathcal{J}(w) = \mathbb{E}[G_1]$ is the altitude of the grasshopper, and our objective is to maximise it.

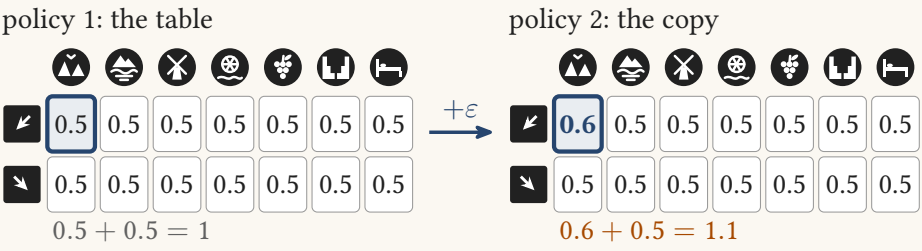
We pick one cell of the table at random



1 Pick one cell of the table at random.

2 The chosen cell plays the grasshopper's part: its number is the spot where the grasshopper stands.

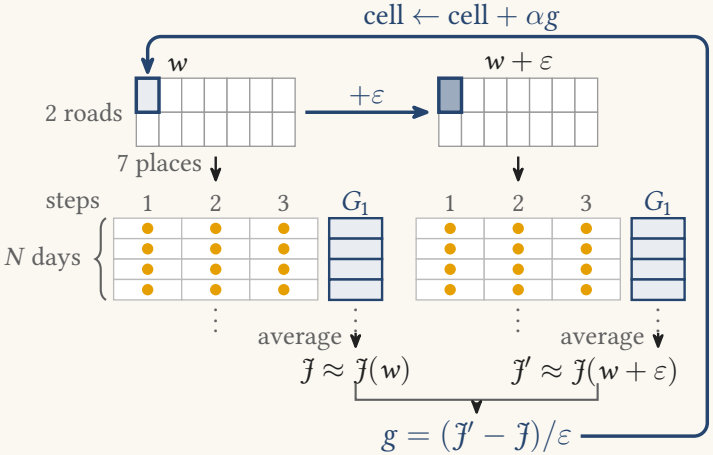
A copy of the table has ε added to the cell



1 Policy 2 is the table with a small number $\varepsilon = 0.1$ added to the chosen cell.

2 In the copy that column adds up to 1.1. We fix this in a moment.

The method in one picture

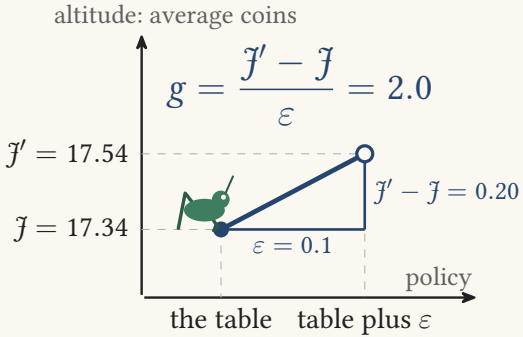


1 The table, and a copy with ε added to the chosen cell.

2 Both play N days with the same dice. G_1 is the coins of a day; J and J' are the averages.

3 The averages give the measured gradient g. The cell jumps by αg and the loop starts again.

Two averages over N days measure the gradient

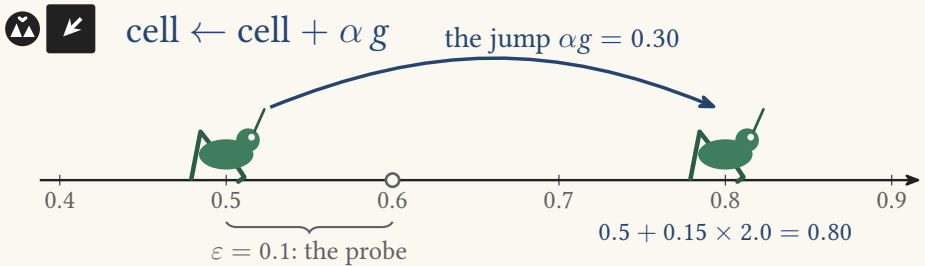


1 Play $N = 100$ days with the table. Dice decide the roads and the luck of each place. Their average, $\mathcal{J} = 17.34$, estimates the altitude $\mathcal{J}(w)$.

2 Play the same days again, with the same dice, on the table plus $\varepsilon = 0.1$ in the chosen cell: $\mathcal{J}' = 17.54$.

3 The same dice give both policies the same luck, so only the change shows. The measured gradient: $g = (\mathcal{J}' - \mathcal{J})/\varepsilon = 2.0$.

The cell jumps by α times the gradient



1 If the copy earned more, g is positive and the cell goes up; otherwise it goes down.

2 α is how far we jump per unit of gradient.

3 ε and α are two different small numbers.

The whole method is a short loop

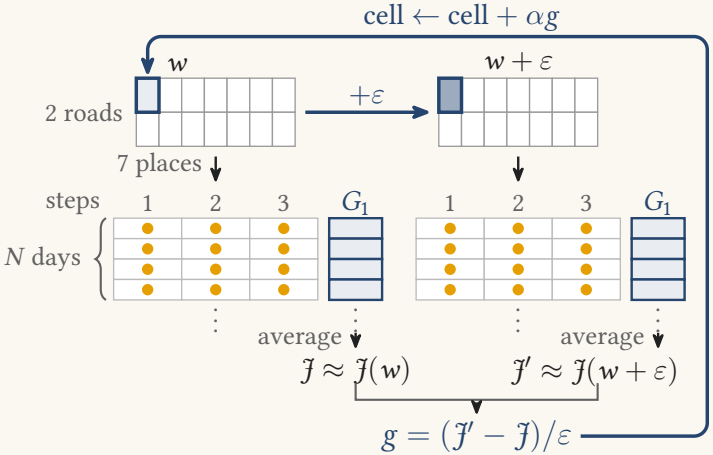
```
repeat:
  pick a cell of the table at random
  make a copy, add eps to that cell
  play N days with each, same dice
  J, J' = average coins of each
  g = (J' - J) / eps
  cell = cell + alpha * g
```

1 The chosen cell is the grasshopper.

2 Two policies, and their two altitudes.

3 The measured gradient, and the jump.

The method in one picture

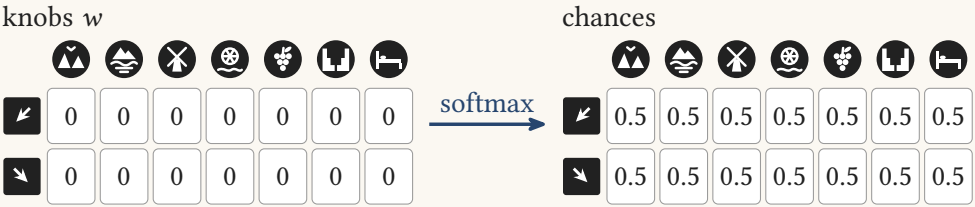


1 The table, and a copy with ε added to the chosen cell.

2 Both play N days with the same dice. G_1 is the coins of a day; J and J' are the averages.

3 The averages give the measured gradient g. The cell jumps by αg and the loop starts again.

Knobs fix the problem: any number is allowed

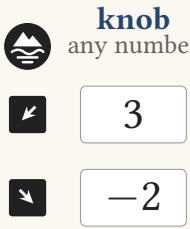


1 Adding ε to a chance can break a column: it must add up to 1.

2 So every cell holds a knob: any number. All knobs start at 0.

3 Turn the knobs, then convert each column into chances: the softmax.

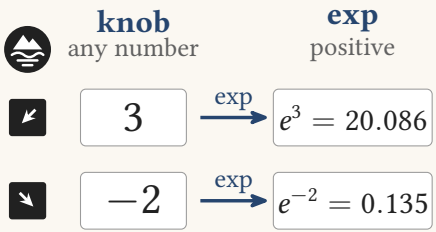
A bigger knob favours its road



1 One column of the table: a knob for each road at one place. A knob is any number, positive or negative.

2 A bigger knob favours its road. Here ↙ is favoured. Two steps turn the knobs into chances.

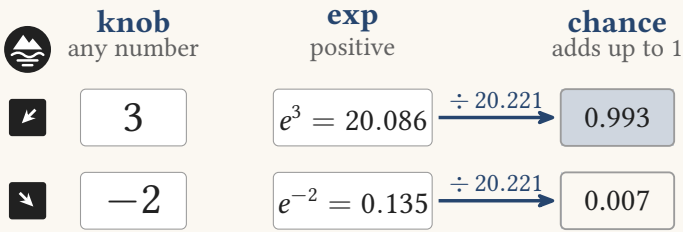
Step 1: take the exponential of each knob



1 The exponential e^x turns any number into a positive one.

2 A large knob becomes a large number. A negative knob becomes a number close to 0.

Step 2: divide each by the sum to get chances



1 Add the two numbers, $20.086 + 0.135 = 20.221$, and divide each by this sum.

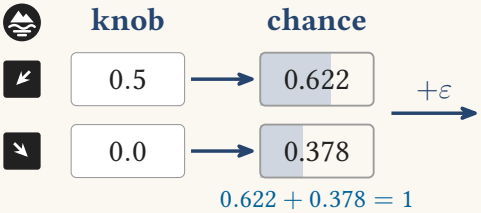
2 The chances add up to 1. Knobs 3 and −2 make the road nearly certain.

$$\text{chance}_i = \frac{e^{w_i}}{e^{w_1} + e^{w_2}}$$

the softmax of a column of two knobs

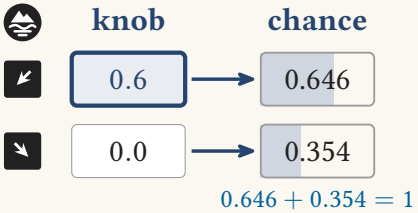
Adding ε to a knob always leaves a valid column

before



$+\varepsilon$

after $\varepsilon = 0.1$ is added to the first knob



1 The first road becomes a little more likely, and the second a little less.

2 Whatever the knobs, a column adds up to 1 and each chance stays between 0 and 1.

The same loop turns the knobs

```
w = knobs, all 0
repeat:
   pick a cell of w at random
   w2 = w, add eps to that cell
   play N days with softmax(w)
   play N days with softmax(w2)
   (the same dice for both)
   J, J' = average coins of each
   g = (J' - J) / eps
   w[cell] = w[cell] + alpha * g
```

1 The table of knobs is where the grasshopper stands.

2 One pass of the loop is one hop of the grasshopper.

3 The softmax turns every column of knobs into chances.

3

Part 3

Policy gradient methods

The table of all gradients is called $\nabla J(w)$

knobs w, for example

↙	1.1	−0.4	0.3	0.7	1.0	−0.2	0.5
↘	0.2	0.8	−0.9	0.7	−0.3	0.6	−0.6

slopes of J

gradient $\nabla J(w)$

↙	+0.64	−0.23	+0.14	+0.04	+0.28	−0.24	+0.01
↘	−0.64	+0.23	−0.14	−0.04	−0.28	+0.24	−0.01

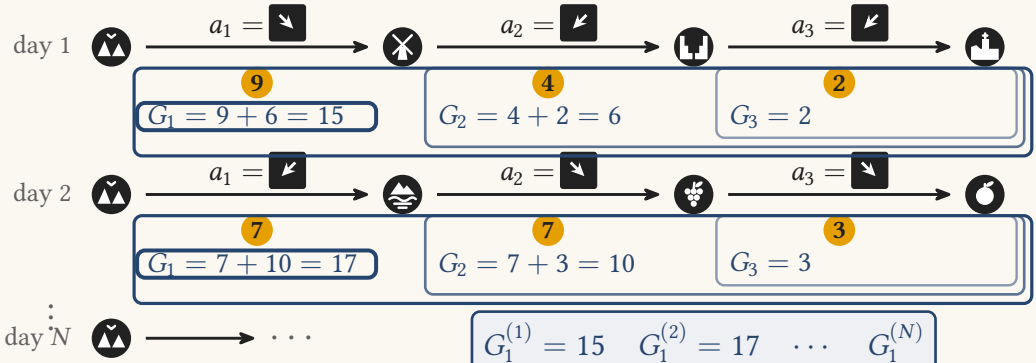
1 These knobs w give $J(w) = 19.03$ coins a day on average.

2 Each knob has a gradient, the slope of J in that knob. The table of all 14 is called $\nabla J(w)$.

3 The loop of the last frames measured one entry at a time.

the entry of road a at place s: $\partial J / \partial w_{a,s}$

A batch of N days has a row for each day



3 The G_1 of each day is its total coins. Their average estimates $\mathcal{J}$.

$\mathcal{J} = \frac{1}{N} \sum_{n=1}^{N} G_1^{(n)} = 17.34$

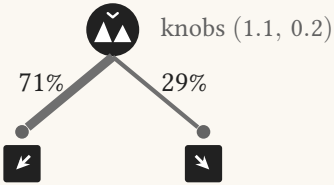
for $N = 100$ days. $\mathcal{J} \approx \mathcal{J}(w)$: our estimate of the altitude.

A formula estimates the whole table from N days

$$\nabla J(w) \approx \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T} \boxed{G_t^{(n)}} \nabla_w \log \pi_w(a_t^{(n)} \mid s_t^{(n)})$$

1. Play N days with π_w, each of $T = 3$ steps. Step t starts at place s_t and takes road a_t.
2. With N days it is an estimate. It settles as N grows, like the average coins did.

The score says how to turn the knobs



1 At the pass the knobs make the chances 71% and 29%.

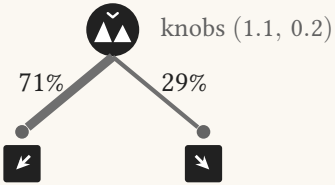
2 Mara takes ↙ (71%). Its score says how to turn the knobs to make ↙ likelier.



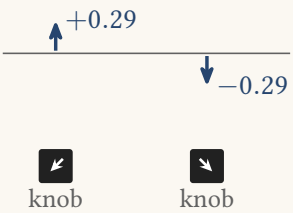
the other 12 knobs of the table get 0

3 The entry of road b is $\mathbf{1}[b = a_t] - \pi_w(b \mid s_t)$: the knob of the road taken goes up, the other knob goes down by as much.

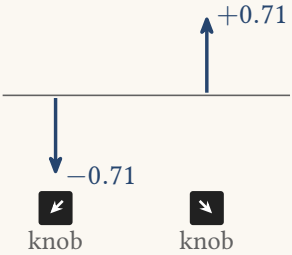
A rare road has a large score



1 Mara takes ↙ (71%): a small score



2 Mara takes ↘ (29%): a large score



3 The rarer the road taken, the larger the score: a rare road moves its knob far.

The gradient averages multiplier times score

average over the N days

multiplier: the coins from step t on

$$\nabla \mathcal{J}(w) \approx \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T} G_t^{(n)} \nabla_w \log \pi_w(a_t^{(n)} \mid s_t^{(n)})$$

one entry for each road a and place s

sum over the steps

score of the road a_t taken at place s_t

1 $\nabla \mathcal{J}(w)$ says how much to adjust the knobs w to increase $\mathcal{J}$.

2 It is an average that grows with G_t and with the score of the road taken.

The multiplier G_t can be replaced by Ψ_t

$$\nabla \mathcal{J}(w) \approx \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T} \boxed{G_t^{(n)}} \nabla_w \log \pi_w(a_t^{(n)} \mid s_t^{(n)})$$

↓ replace G_t by Ψ_t

$$\nabla \mathcal{J}(w) \approx \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T} \boxed{\Psi_t^{(n)}} \nabla_w \log \pi_w(a_t^{(n)} \mid s_t^{(n)})$$

1. It has been shown that the formula still works with another multiplier, written Ψ_t.
2. There are several options for Ψ_t. The first is the one we have: $\Psi_t = G_t$.

4

Part 4

REINFORCE

$\Psi_t = G_t$ gives REINFORCE

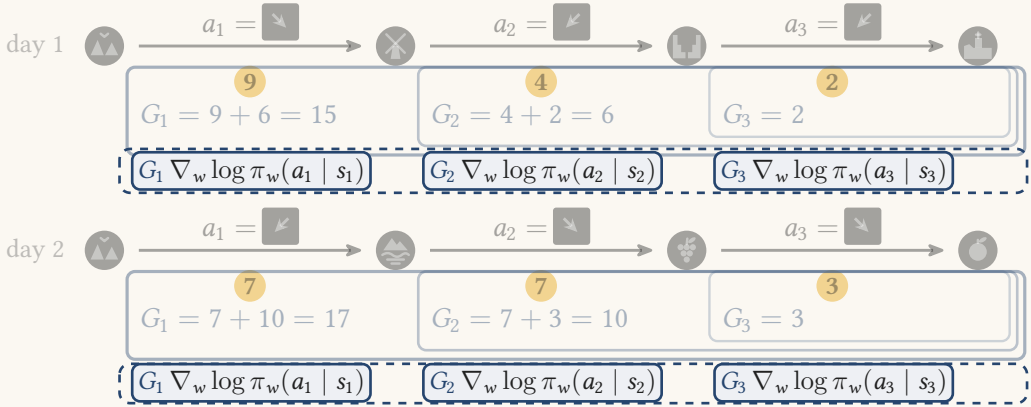
$$w \leftarrow w + \alpha \underbrace{\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T}}_{2} \underbrace{\boxed{G_t^{(n)}}}_{3} \nabla_w \log \pi_w(a_t^{(n)} \mid s_t^{(n)})$$

1 Gradient ascent again: move the knobs by α times the gradient.

2 The average over N days and $T = 3$ steps is the estimate of the gradient of the last frames.

3 The multiplier's box holds G_t, the coins from step t to the end of the day: the first option of the last frame.

One term of the gradient for each cell



3 Sum each row, average the rows: $\nabla J(w) \approx \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{3} G_t^{(n)} \nabla_w \log \pi_w(a_t^{(n)} \mid s_t^{(n)})$

REINFORCE is a short loop

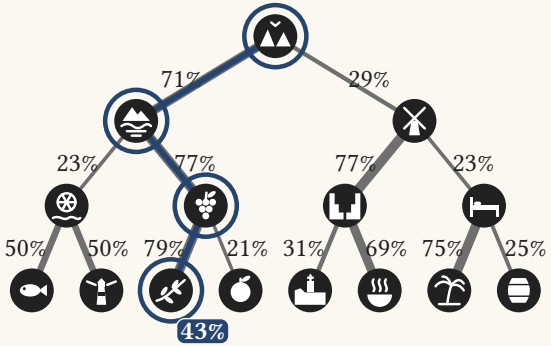
```
w = knobs, all 0
repeat:
1   play N days with softmax(w)
    push every road of every day
2     as hard as the coins after it
3   w = w + alpha * average push
```

1 N days with the current knobs. No copy of the table and no ε.

2 Every road taken is pushed as hard as G_t, the coins that came after it.

3 The knobs move by α times the average push.

Mara’s knobs make the 21-coin day likely

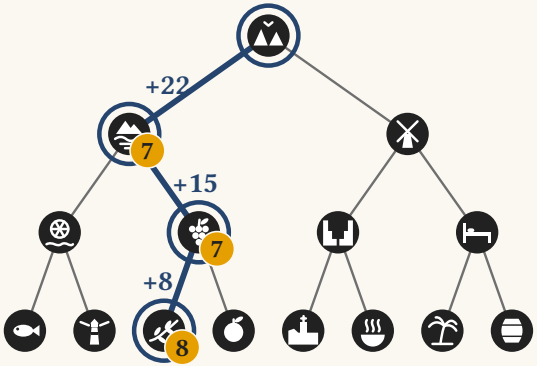


1 Each road is as thick as its chance. The numbers are the chances of the two roads of a place.

2 These knobs make the best day likely: the path that pays 21 coins on average is taken on 43 days in 100.

3 Their average day pays $\mathcal{J}(w) = 19.03$ coins. A few days will move the knobs.

Push each road by the coins that came after it

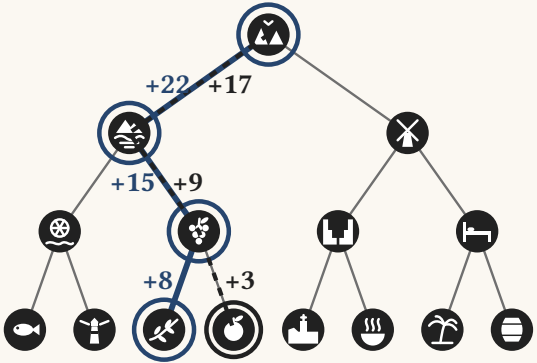


1 One day:  →  →  . It was paid $7 + 7 + 8 = 22$ coins, luck included.

2 Each road is pushed up by the coins collected from it on: 22, then 15, then 8.

3 A road gets credit for what came after it. The coins collected before it do not count.

A richer day pushes the shared roads harder

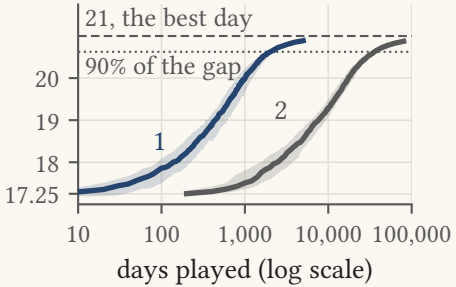


1 A second day (dashed): → → . It was paid $8 + 6 + 3 = 17$ coins: pushes of 17, 9 and 3.

2 The days share two roads. The richer day pushes them harder: 22 against 17, 15 against 9.

3 At they split: is pushed by 8, by 3.

REINFORCE learns from days alone



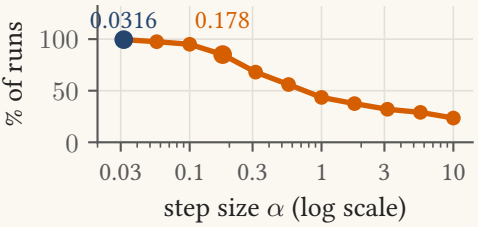
Median of 200 runs and their middle half. $\alpha = 0.0316$ is the fastest step at which 99 of 100 other runs close 90% of the gap.

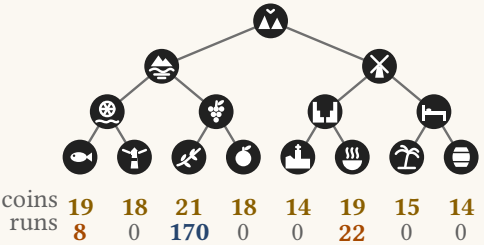
1 REINFORCE (10 days an update, $\alpha = 0.0316$) closes 90% of the gap from 17.25 to 21 coins in a median of 1,950 days.

2 Part 2's method ($N = 100$, $\varepsilon = 0.1$, $\alpha = 0.5$) needs 35,800 days: 200 days played for every move of one cell. Tuned for cost ($N = 5$, $\varepsilon = 1$, $\alpha = 1$) it needs about 900.

3 One update of REINFORCE moves 13 of the 14 knobs, from 10 days. One update of Part 2's method moves one, from 200 days.

A larger step makes runs fail



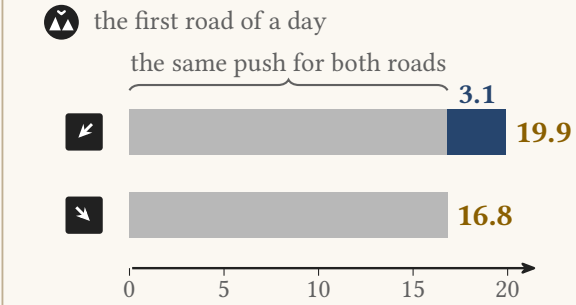


1 At $\alpha = 0.0316$, 199 of 200 runs close 90% of the gap. At $\alpha = 0.178$, 5.6 times larger, 170 do.

2 The other 30 runs end on a day that pays 19 coins. The best day pays 21.

3 Every road taken is pushed up, so with a large step the first roads taken can run away: the runs that fail settle on their day after about 120 days, like the others.

Every road taken goes up, whatever it pays

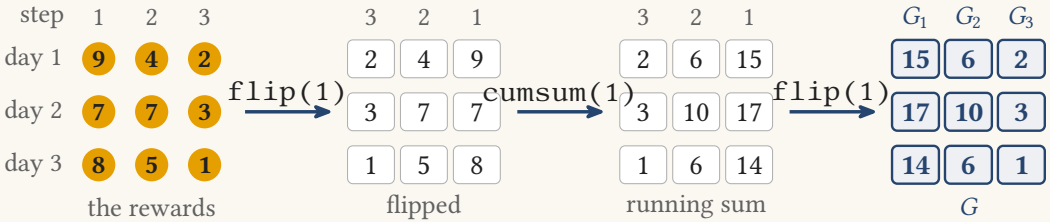


1 Days that start with ↙ pay 19.9 coins on average, days that start with ↘ 16.8.

2 Each day pushes the road it took by its coins. Coins are never negative, so both roads go up.

3 Only the difference, 3.1 coins, tells the better road apart. It is small next to the pushes, and a batch of days can hide it.

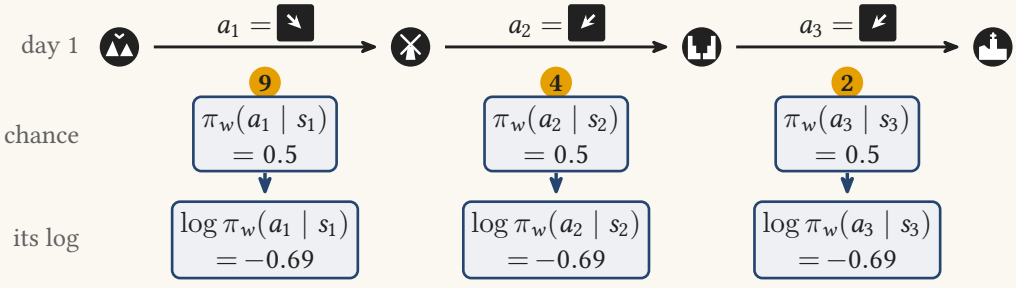
G is a running sum from the last step back



4 Flip back: the cell of step t is G_t, the sum of the rewards from step t on. The 1 means along each row, the steps.

```
G = rewards.flip(1).cumsum(1).flip(1)
```

$\log \pi$ is the log of the chance of each road taken

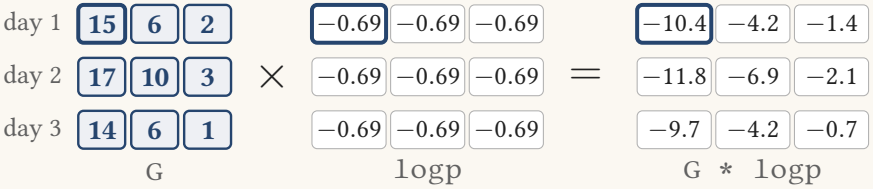


step	1	2	3
day 1	−0.69	−0.69	−0.69
day 2	−0.69	−0.69	−0.69
day 3	−0.69	−0.69	−0.69

```
logp = dist.log_prob(roads)
```

4 The three days make the tensor `logp`, a row for each day and a column for each step.

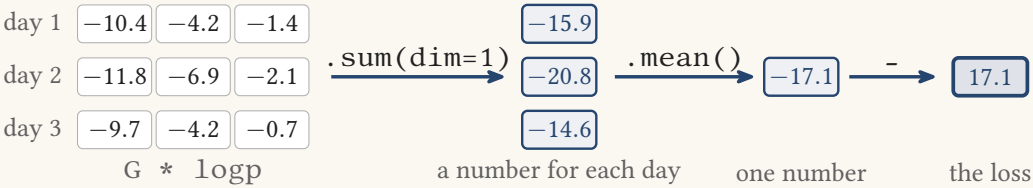
The loss multiplies G and $\log \pi$ cell by cell



first cell: $G_1 \times \log \pi_w(a_1 \mid s_1)$: $\quad 15 \times (-0.69) = -10.4$

3 The product, cell by cell, is a tensor of the same shape: one number for each step of each day.

Add up the steps, average the days, change the sign



4 Change the sign: this is the loss. Its gradient is minus the average of G_t times the score, so a step downhill on it is a step uphill on $\mathcal{J}$.

```
loss = -(G * logp).sum(dim=1).mean()
```

A library writes the same update in four lines

```
loss = -(G * logp).sum(dim=1).mean()
optimiser.zero_grad()
loss.backward()
optimiser.step()
```

1 `G` and `logp` hold G_t and $\log \pi_w(a_t \mid s_t)$: one row for each of the N days, one column for each of the T steps. The loss sums each row, averages the days and flips the sign.

2 `backward` gives the gradient of the loss: minus the average of G_t times the score.

3 A step downhill on the loss is a step uphill on $\mathcal{J}$: the update of the first frame.